

AI 코딩 어시스턴트 도입에 따른 리스크 관리 전략

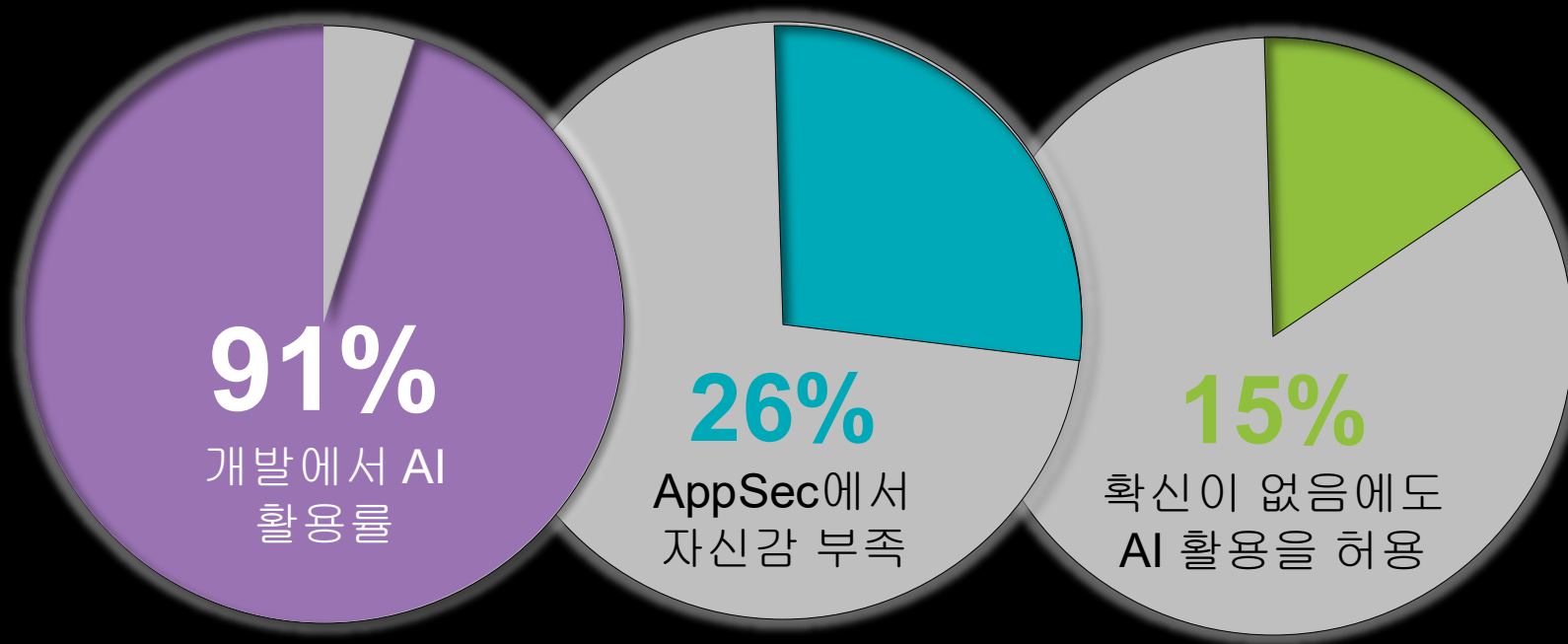
Black Duck
Sales Engineer 정성훈 부장



소프트웨어 개발 분야에서의 AI 혁명

급변하는 환경 속에서 혁신과 리스크

AI 기반 소프트웨어 개발의 확장이
위험 노출을 **증폭**시키고 있습니다.



자신감이 부족함에도 불구하고 코딩 어시스턴트가 널리 사용되고 있습니다.



AI 코딩 어시스턴트는 개발 속도와 코드 양에만 영향을 미치는 것이 아닙니다.



보안

- 결함 코드
- 취약한 종속성
- 부적절한 데이터 처리
- 새롭게 등장하는 AI 모델 벡터



라이선스

• OSS 컴포넌트 라이선싱
• AI 모델 라이선싱

- AI 생성 코드에 대한 권리
- “숨겨진” 코드 조각



신뢰

- AI 코딩 어시스턴트에 대한 암묵적 신뢰
- 불투명한 AI 모델/학습
- 책임 전가
- 파급 효과

AI 코딩 어시스턴트는 개발 속도와 코드 양에만 영향을 미치는 것이 아닙니다.



보안

보안이 검증되지 않은 방대한 데이터로 학습

- 결함 코드, 부적절한 입력 검증, 취약한 종속성

AI는 보안이 확보된 코드 보다는 기능 구현이 된 코드에 우선 순위를 부여

문제가 조직 전체에 복제 되거나 확산 될 수 있음

오픈 소스 AI 모델은 새로운 공격 경로 제공

- *Prompt injection, Data and model poisoning, Prompt leakage, Improper output handling*



라이선스



신뢰

많은 팀이 AI 생성 코드를 보호할 준비가 되어 있지 않습니다.

AI 생성 코드 보안에 대한
자신감 부족

AI 코드 생성기가 잘 피하는
취약점:

Out of bounds Write

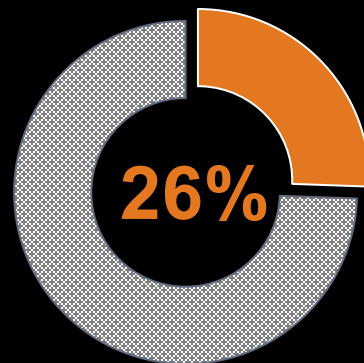
CWE 79 - Cross Site Scripting

CWE 416 - Use After Free

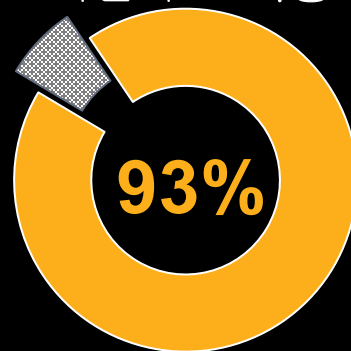
CWE 125 - Out of Bounds Read

CWE 190 - Integer Overflow

CWE 119 - Improper Restriction of
Operations



AppSec 자신감 부족에도
여전히 AI 사용



AI 코드 생성기에서 발생하기
쉬운 취약점:

- CWE 20 - Improper Input Validation
- CWE 502 - Deserialization of Untrusted Data
- CWE 78 - OS Command Injection
- CWE 22 - Path Traversal
- CWE 434 - Unrestricted Upload of File with Dangerous Type
- CWE 522 - Insufficiently Protected Credentials

인공지능이 항상 완벽한 것은 아니다.

현재 GitHub에 있는 Copilot 생성 코드의
약 35%에는 보안 취약점이 있음

*Security Weaknesses
of Copilot Generated Code in GitHub*
October 2023

Copilot은 취약한 코드를 33%의 확률로 그대로
재현했고, 25%는 수정했으며, 42%는 전혀 다른
방식으로 작성!

*Is GitHub's Copilot as Bad as Humans
at Introducing Vulnerabilities in Code?*
August 2023

Is GitHub's Copilot as Bad as Humans at

Security Weaknesses of Copilot Generated Code in GitHub

Yujia Fu
School of Computer Science
Wuhan University
Wuhan, China
yujia_fu@whu.edu.cn

Peng Liang
School of Computer Science
Wuhan University
Wuhan, China
liangp@whu.edu.cn

Amjed Tahir
School of Mathematical and
Computational Sciences
Massey University
Palmerston North, New Zealand
a.tahir@massey.ac.nz

Zengyang Li
School of Computer Science
Central China Normal University
Wuhan, China
zengyangli@ccnuc.edu.cn

Mojtaba Shahin
School of Computing Technologies
RMIT University
Melbourne, Australia
mojtaba.shahin@rmit.edu.au

Jiaxin Yu
School of Computer Science
Wuhan University
Wuhan, China
jiaxin.yu@whu.edu.cn

ABSTRACT

Modern code generation tools use AI models, particularly Large Language Models (LLMs), to generate functional and complete code. While such tools are becoming popular and widely available for developers, using these tools is often accompanied by security challenges, leading to insecure code merging into the code base. Therefore, it is important to assess the quality of the generated code, especially in terms of its security. Researchers have recently explored various aspects of code generation tools, including security. However, many open questions about the security of the generated code require further investigation, especially the security issues of automatically generated code in the wild. To this end, we conducted an empirical study by analyzing the security weaknesses in code snippets generated by GitHub Copilot that are listed as part of publicly available projects hosted on GitHub. The goal is to investigate the types of security issues and their scale in real-world scenarios (rather than crafted scenarios). To this end, we identified 435 code snippets generated by GitHub Copilot from publicly available projects. We then conducted extensive security analysis to identify Common Weakness Enumerations (CWEs) instances in these code snippets. The results show that (1) 50.8% of Copilot-generated code snippets contain CWEs, and these issues are spread across multiple languages; (2) the security weaknesses are diverse and related to 42 different CWEs, in which CWE-38 OS Command Injection, CWE-359 Use of Insufficiently Random Values, and CWE-780 Improper Check or Handling of Exceptional Conditions occurred the most frequently; and (3) among the 42 CWEs identified, 11 of them belong to the currently recognized 2022 CWE Top-25. Our findings confirm that developers should be careful when editing code generated by Copilot (and similar AI code generation tools) and should also run appropriate security checks as they accept the

suggested code. It also shows that practitioners should cultivate corresponding security awareness and skills.

CCS CONCEPTS

• Software and its engineering → Software development tools; require; • Security and privacy → Software security engineering

KEYWORDS

Code Generation, Security Weaknesses, CWEs, GitHub Copilot

ACM Reference Format

Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, and Jiaxin Yu. 2023. Security Weaknesses of Copilot Generated Code in GitHub. In *Proceedings of ACM Conference (Conference '23)*, 2678, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3555555>

1 INTRODUCTION

Code generation tools aim to automatically generate functional code based on prompts, which can include text descriptions (comments), code (such as function signatures, expressions, variable names, etc.), or a combination of text and code [34]. After writing an initial code or comment, developers can rely on code generation tools to complete the remaining code. This approach can save development time and accelerate the software development process. Automated code generation tools have always been an active research discussion topic [22, 34]. Some of the earliest work can be traced back to the 1960s when Waldinger and Lee proposed a program synthesizer called PROLOG, which automatically generated LISP programs based on specifications provided by users in the form of predicate calculus [46]. As computer languages continued to evolve, more and more programming languages began to support macros programming, making automated code generation technology more efficient and flexible. In recent years, the rapid development of artificial intelligence technologies, particularly in the form of machine learning and deep learning models, has accelerated the development of code generation technologies.

Recent advancements in code generation come with the emergence of Large Language Models (LLMs). LLMs are deep learning models trained on a large dataset corpus with powerful language understanding capabilities that can be used for tasks such as natural

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for this work is held by the author(s). All rights reserved. Reproduction, distribution, and sale in any form is prohibited without the express written permission of the publisher. For more information, please contact the publisher at permissions@blackduck.com.
Copyright © 2023, by the author(s).
This is a preprint of the article published in *Proceedings of ACM Conference (Conference '23)*, 2678, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3555555>

AI 코딩 어시스턴트는 개발 속도와 코드 양에만 영향을 미치는 것이 아닙니다.



보안



라이선스



신뢰

출처 표시 없이 라이선스가 있는 소스에서 생성된 AI 코드 (*GPL, AGPL, etc.*)

출처 추적 및 컴플라이언스 관리의 복잡성

AI 생성 코드 저작권에 대한 커져가는 논쟁

AI가 생성한 코드가 항상 당신의 것은 아닙니다.

```
Scanned File

github-copilot-generated.c

Scanned File Path: file:///Users/kadu/Downloads/github-copilot-snippet-scan/github-copilot-generated.c
File Size: 3.83 KB
(Line 88 to 99)

86 }
87
88 /* allocate a sparse matrix (triplet form or compressed-column form) */
89 cs *cs_spalloc (int m, int n, int nzmax, int values, int triplet)
90 {
91     cs *A = (cs *) cs_calloc (1, sizeof (cs)) ; /* allocate the cs struct */
92     if (!A) return (NULL) ; /* out of memory */
93     A->m = m ; /* define dimensions and nzmax */
94     A->n = n ;
95     A->nzmax = nzmax = CS_MAX (nzmax, 1) ;
96     A->nz = triplet ? 0 : -1 ; /* allocate triplet or comp.col */
97     A->p = (int *) cs_malloc (triplet ? nzmax : n+1, sizeof (int)) ;
98     A->i = (int *) cs_malloc (nzmax, sizeof (int)) ;
99     A->x = values ? (double *) cs_malloc (nzmax, sizeof (double)) : NULL ;
100     return ((!A->p || !A->i || (values && !A->x)) ? cs_done (A, NULL, NULL, 0)
101             : A) ;
102 }
103
104
105 |
```

- GPT는 수백만 줄의 오픈 소스로 학습
- 생성된 코드는 오픈소스 코드의 “코드 조각”을 포함
- 라이선스와 저작권을 준수하지 않으면 팀이 법적 위험에 처할 수 있음

AI가 생성한 코드가 항상 당신의 것은 아닙니다.

Scanned File

github-copilot-generated.c

Scanned File Path: file:///Users/kadu/Downloads/github-copilot-snippet-scan/github-copilot-generated.c

File Size: 3.83 KB

Matched Component

acado last_1_1_0_beta

License: GNU Lesser General Public License v3.0 only

Release Date: Oct 6, 2013

Matched File Path: /acado-e2ba77daa5171e589c105c794cbf23c4c8385dea/external_packages/csparse/cs_util.c

Snippet Match: 17%

(Line 2 to 13)

```
1 #include "cs.h"
2 /* allocate a sparse matrix (triplet form or compressed-column form) */
3 cs *cs_spalloc (int m, int n, int nzmax, int values, int triplet)
4 {
5     cs *A = (cs*) cs_calloc (1, sizeof (cs)) ; /* allocate the cs struct */
6     if (!A) return (NULL) ; /* out of memory */
7     A->m = m ; /* define dimensions and nzmax */
8     A->n = n ;
9     A->nzmax = nzmax = CS_MAX (nzmax, 1) ;
10    A->nz = triplet ? 0 : -1 ; /* allocate triplet or comp.col */
11    A->p = (int*) cs_malloc (triplet ? nzmax : n+1, sizeof (int)) ;
12    A->i = (int*) cs_malloc (nzmax, sizeof (int)) ;
13    A->x = values ? (double*) cs_malloc (nzmax, sizeof (double)) : NULL ;
14    return ((!A->p || !A->i || (values && !A->x)) ? cs_sprealloc (A) : A) ;
15 }
16
17 /* change the max # of entries sparse matrix */
18 int cs_sprealloc (cs *A, int nzmax)
19 {
20     int ok, oki, okj = 1, okx = 1 ;
```

acado last_1_1_0_beta Component License

Attribution Statement

License Edit

Select a license to view, Toggle Edit Mode on to edit.

GNU Lesser General Public License v3.0 only

GNU Lesser General Public License v3.0 only

Status: Unreviewed | Family: Weak Reciprocal

Required	Forbidden	Permitted
> Distribute Original	> Patent Retaliation	> Use Patent
> Right to Copy	> Hold Liable	> Commercial
> Include Copyright	> Anti DRM Provision	> Distribute
> Reverse Engineer	> Fees	> Place War
> State Changes	> Sub-License	> Modify
> Disclose Source		

BLACK DUCK

© 2025 Black Duck Software, Inc.

AI 코딩 어시스턴트는 개발 속도와 코드 양에만 영향을 미치는 것이 아닙니다.



보안



라이선스



신뢰

AI 코딩 어시스턴트에 대한 암묵적 신뢰는 신중함을 무디게 하며 책임을 전가

- 명확하지 않은 AI 모델과 훈련 셋
- AI를 지나치게 신뢰하는 주니어 개발자들의 태도는 AppSec 테스트로는 드러나지 않는 문제를 초래
- 성능문제, 런타임 오류

개발자가 늘어날 수록 그 결과의 영향력은 더욱 커집니다.

전략: AI 기반 파이프라인에서 자동화된 보안 체계

더 빠른 개발과, 안전한 배포

AI 중심 개발 환경에 최적화된 파이프라인 기반 보안 테스트

코드 수정

개발자들의 코드 변경 사항 반영;
AI 생성 코드와 OSS

소프트웨어 구성요소 분석(SCA)

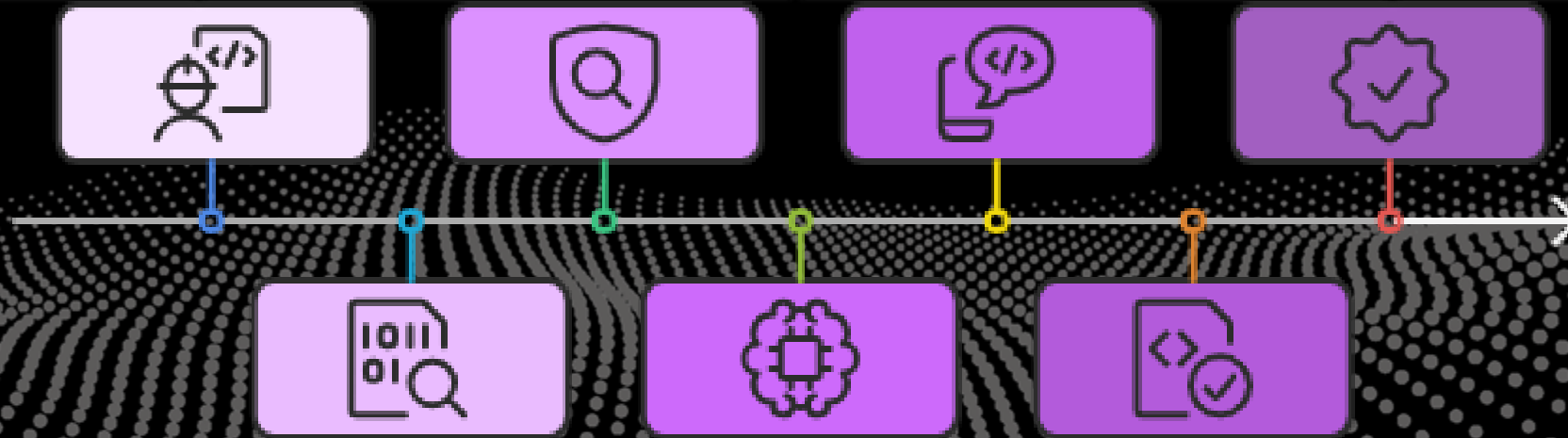
오픈 소스 취약점 탐지를 위한 코드
스캔

수정안의 개발자 전달

LLM이 판단한 수정안이 파이프라인
안에서 개발자에게 제시

애플리케이션 재테스트

검증된 수정사항을 적용한
애플리케이션의 재테스트 및
컴플라이언스 확인



정적 분석(SAST)

CWE와 결함 탐지를 위한 코드 스캔

LLM의 보안 수정안 결정

개선안을 찾기 위해 결함 있는
문제들이 LLM에 의해 분석

검증을 위한 코드 재스캔

개선 확인 및 신규 이슈 방지를 위한
코드 수정 사항 재스캔

AI 코드 생성기의 규모와 속도에 맞춘 보안

코드 수정

개발자들의 코드 변경 사항 반영;
AI 생성 코드와 OSS

소프트웨어 구성요소 분석(SCA)

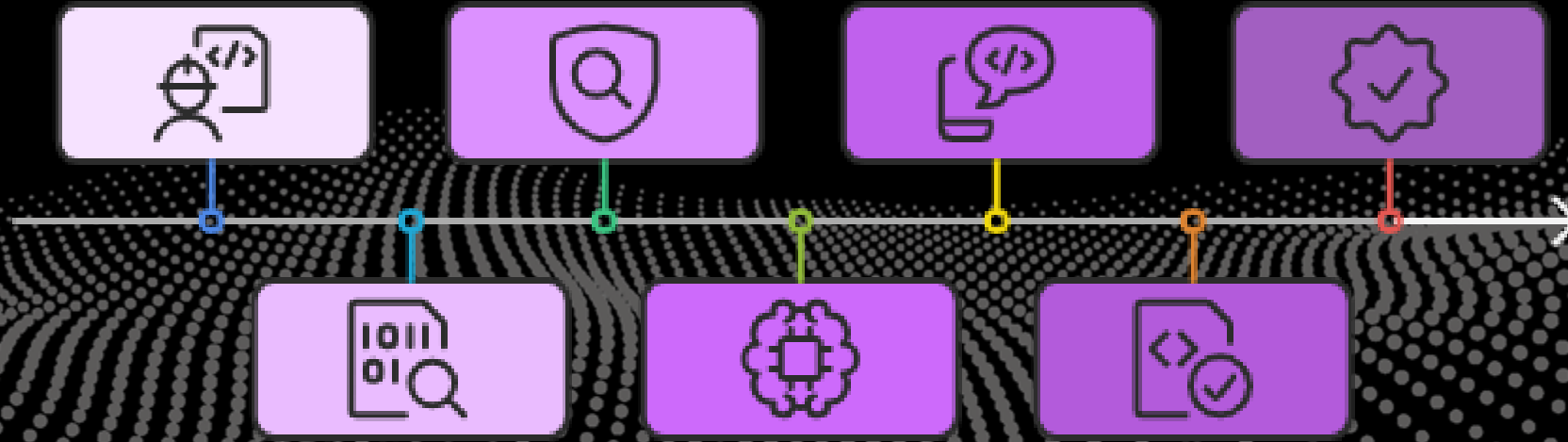
오픈 소스 취약점 탐지를 위한 코드
스캔

수정안의 개발자 전달

LLM이 판단한 수정안이 파이프라인
안에서 개발자에게 제시

애플리케이션 재테스트

검증된 수정사항을 적용한
애플리케이션의 재테스트 및
컴플라이언스 확인



정적 분석(SAST)

CWE와 결함 탐지를 위한 코드 스캔

LLM의 보안 수정안 결정

개선안을 찾기 위해 결함 있는
문제들이 LLM에 의해 분석

검증을 위한 코드 재스캔

개선 확인 및 신규 이슈 방지를 위한
코드 수정 사항 재스캔

AI가 만든 문제를 완화하기 위한 AI 기반의 보안

코드 수정

개발자들의 코드 변경 사항 반영;
AI 생성 코드와 OSS

소프트웨어 구성요소 분석(SCA)

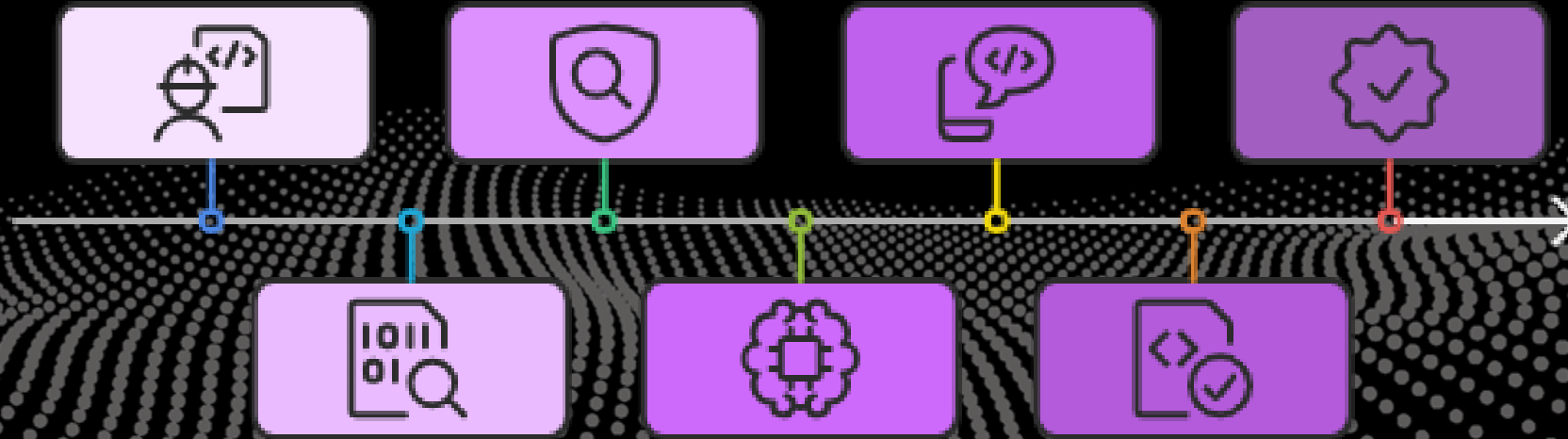
오픈 소스 취약점 탐지를 위한 코드
스캔

수정안의 개발자 전달

LLM이 판단한 수정안이 파이프라인
안에서 개발자에게 제시

애플리케이션 재테스트

검증된 수정사항을 적용한
애플리케이션의 재테스트 및
컴플라이언스 확인



정적 분석(SAST)

CWE와 결함 탐지를 위한 코드 스캔

LLM의 보안 수정안 결정

개선안을 찾기 위해 결함 있는
문제들이 LLM에 의해 분석

검증을 위한 코드 재스캔

해결은 잘 되었고 신규 이슈가
발생하지 않도록 코드 수정 사항을
재스캔

AI가 생성한 코드에서 보안 결함을 탐지하기 위해 정적 분석을 진행하세요.

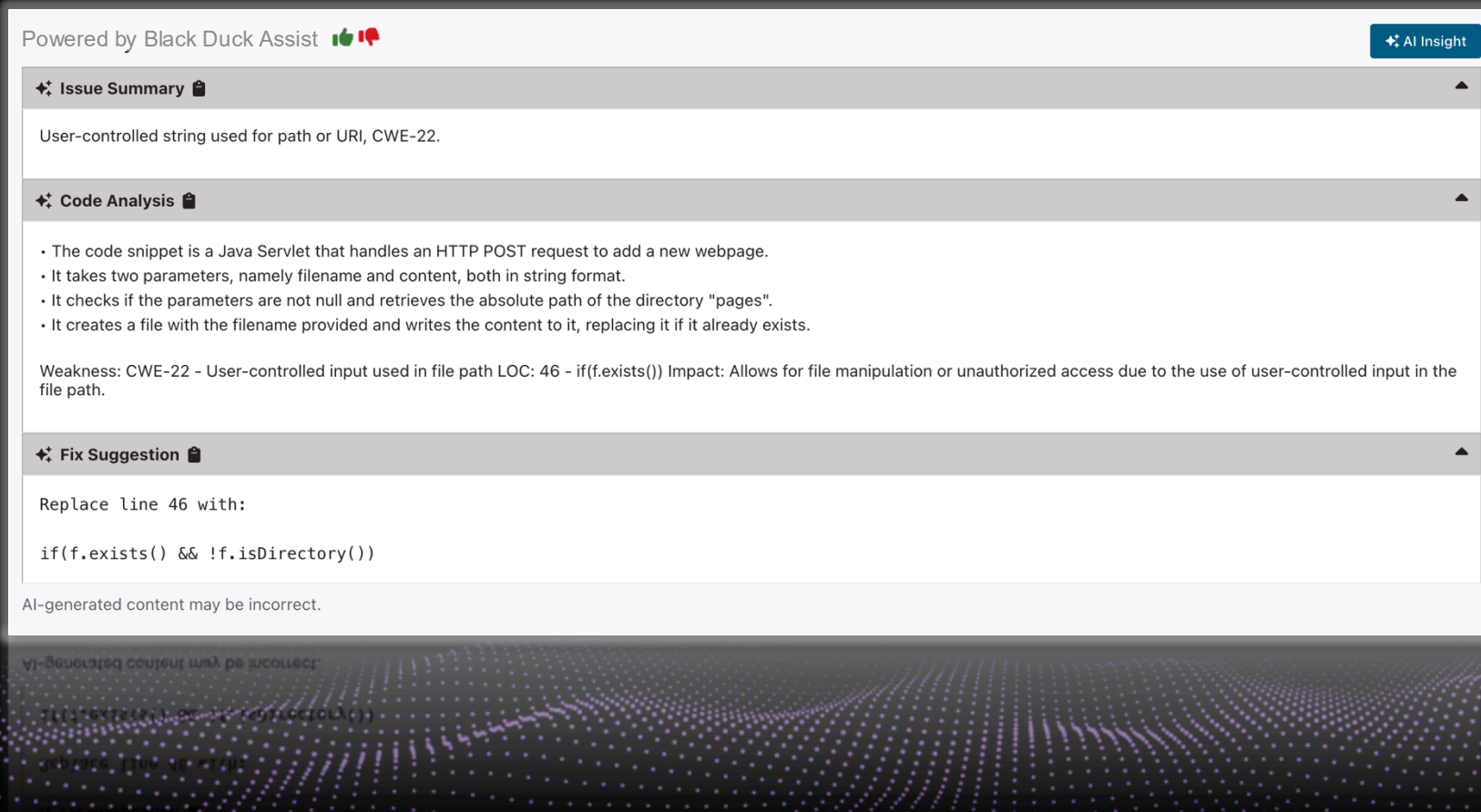
The screenshot displays the Black Duck web interface. The left sidebar shows navigation options like Summary, Issues, Components, Licenses, Tests, Branches, and Settings. The main area is titled 'Issue Details' and 'Contributing Code Events'. It shows a 'Severity: High' issue of type 'Directory Traversal' with 1 occurrence. The location is 'src/main/java/org/cysecurity/cspt/jvl/controller/AddPage.java'. The analysis steps are listed: 1. Calling 'getParameter'. 2. Assigning 'fileName'. 3. Creating a tainted string. 4. Assigning a tainted string to 'filePath'. 5. Calling 'File'. 6. Assigning 'f'. 7. Passing 'f' to 'exists'. 8. Constructing a path or URI using the tainted value in 'f'. 9. Path manipulation vulnerabilities can be addressed by proper input validation. The code snippet shows a Java method with a directory traversal vulnerability.

```
39 String fileName=request.getParameter("filename");
40 String content=request.getParameter("content");
41 if(fileName!=null && content!=null)
42 {
43     String pagesDir=getServletContext().getRealPath("/pages");
44     String filePath=pagesDir+"/"+fileName;
45     File f=new File(filePath);
46     if(f.exists())
47     {
48         f.delete();
49     }
50     if(f.createNewFile())
51     {
```

보안 및 품질 결함을 찾고 수정 하는 이유:

- 결함 있는 코드로 학습됨
- 잘못 처리된 LLM 응답 데이터
- 부실한 프롬프트
- AI 환각 현상

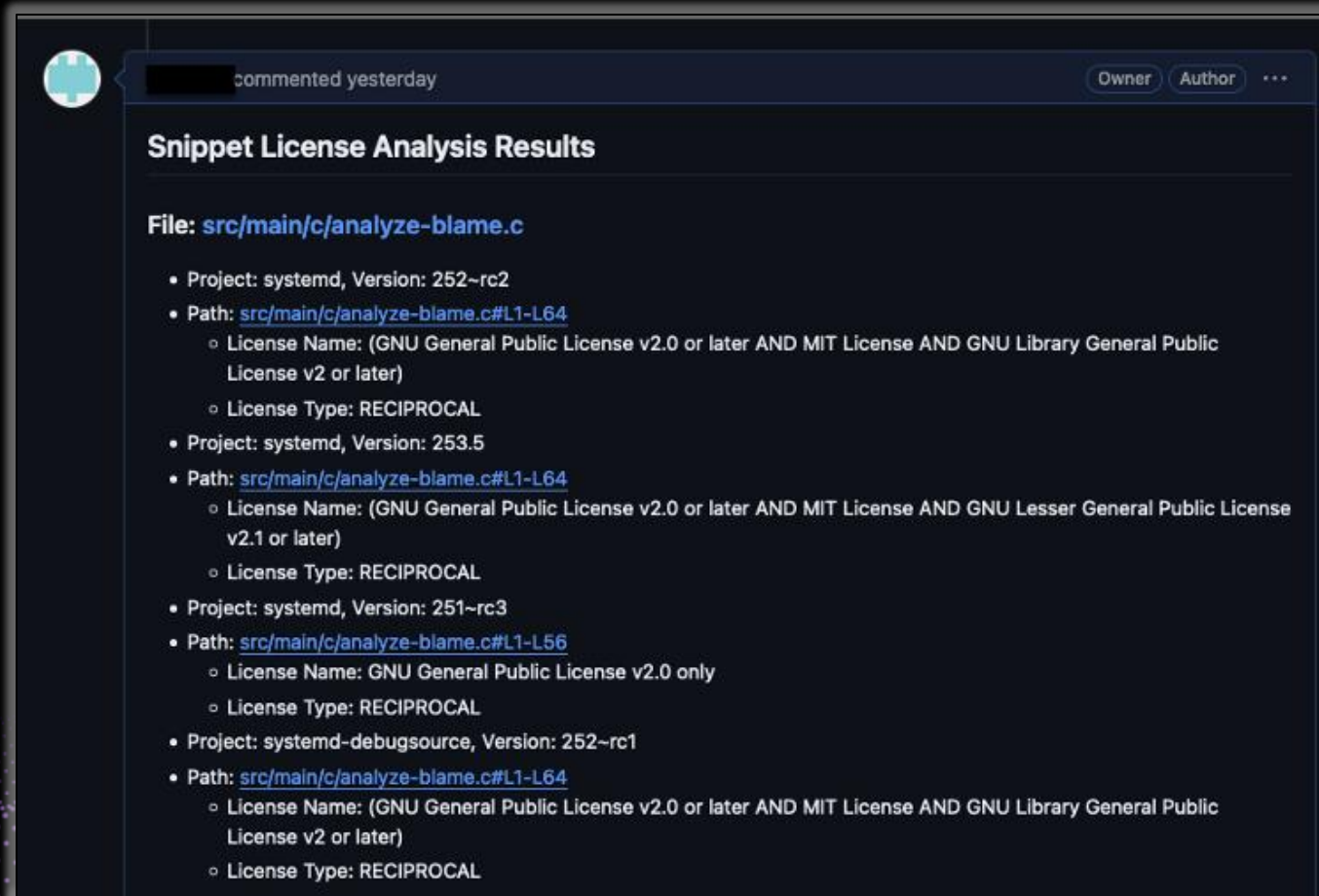
AI가 생성한 코드에서 보안 결함을 탐지하기 위해 정적 분석을 진행하세요.



보안 및 품질 결함을 찾고 수정 하는 이유:

- 결함 있는 코드로 학습됨
- 잘못 처리된 LLM 응답 데이터
- 부실한 프롬프트
- AI 환각 현상

“pull request” 시점에 자동으로 IP 위반 탐지



- AI로 생성된 코드가 커밋 될 때 라이선스 이슈를 식별
- “pull request” 분석의 자동화
- 결과를 “pull request”의 코멘트로 결과 남김

LLM에 맞춰 진화하는 최신 모범 사례에 부합

```
import java.sql.Statement;
import java.util.*;
import com.azure.ai.openai.*;
import com.azure.ai.openai.models.*;
import com.azure.core.credential.AzureKeyCredential;

public class Test {

    public void OpenAITest() {
        OpenAIClient client = new OpenAIClientBuilder()
            .credential(new AzureKeyCredential("{key}"))
            .endpoint("{endpoint}")
            .buildClient();

        List<ChatRequestMessage> chatMessages = new ArrayList<>();

        ChatCompletions chatCompletions = client.getChatCompletions("{deploymentNameOrModelName}",
            new ChatCompletionsOptions(chatMessages));

        Statement stmt;
        for (ChatChoice choice: chatCompletions.getChoices())
        {
            stmt.executeQuery("SELECT * FROM " + choice.getMessage().getContent());
        }; //defect#SQLI
    }
}
```

Supply Chain(공급망)

OWASP LLM-03

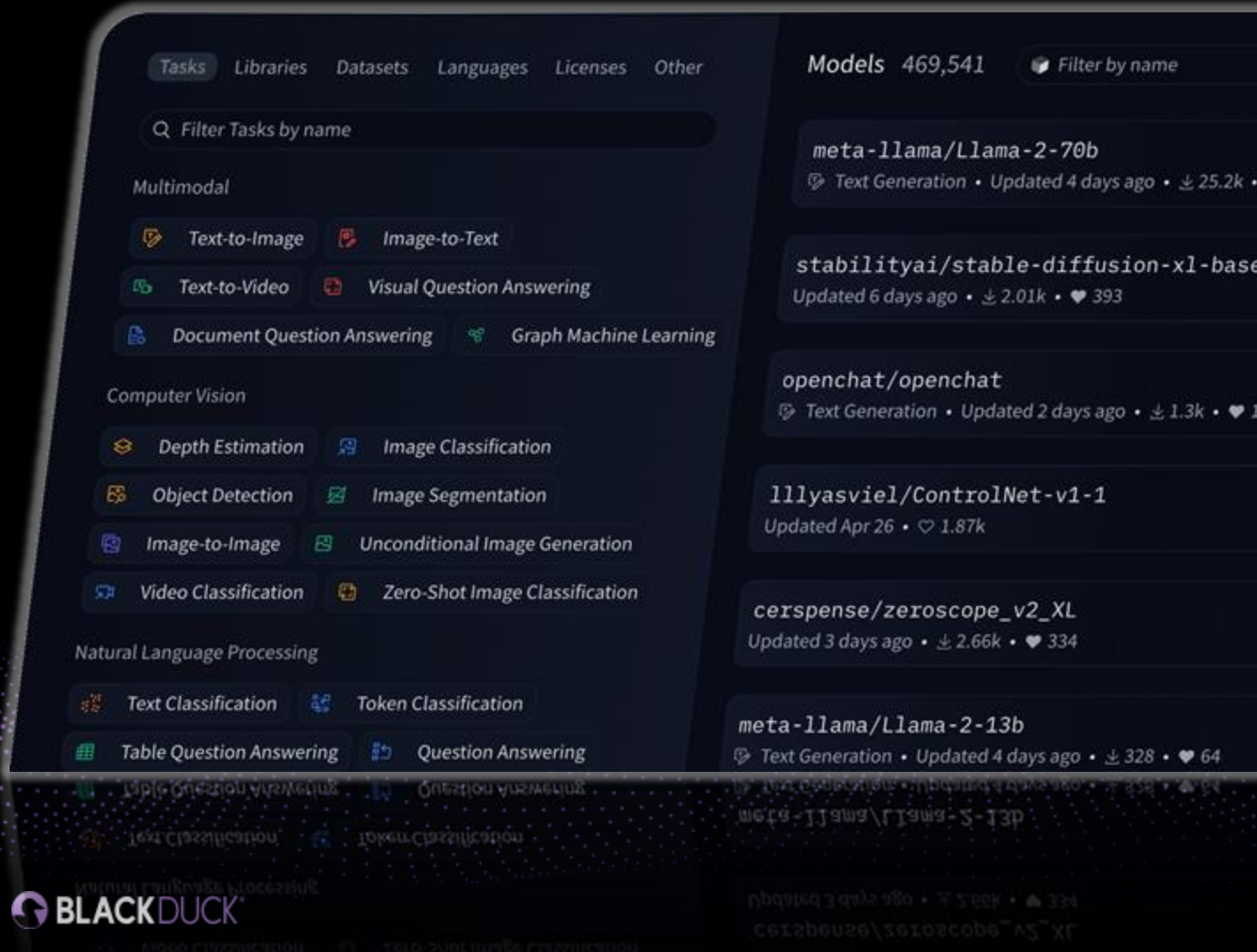
- 특정 LLM들에 대해 신뢰도를 평가하고 등급에 따른 분류
- 편향된 출력, 보안 침해 그리고 시스템 장애를 예방

Improper Output Handling (부적절한 출력 처리)

OWASP LLM-05

- LLM 데이터가 잠재적으로 왜곡되었을 수 있음을 인지
- LLM 응답 데이터가 적절히 검증되고 위험 요소가 제거되었는지 확인

애플리케이션 코드를 넘어 **AI 모델**로 확장

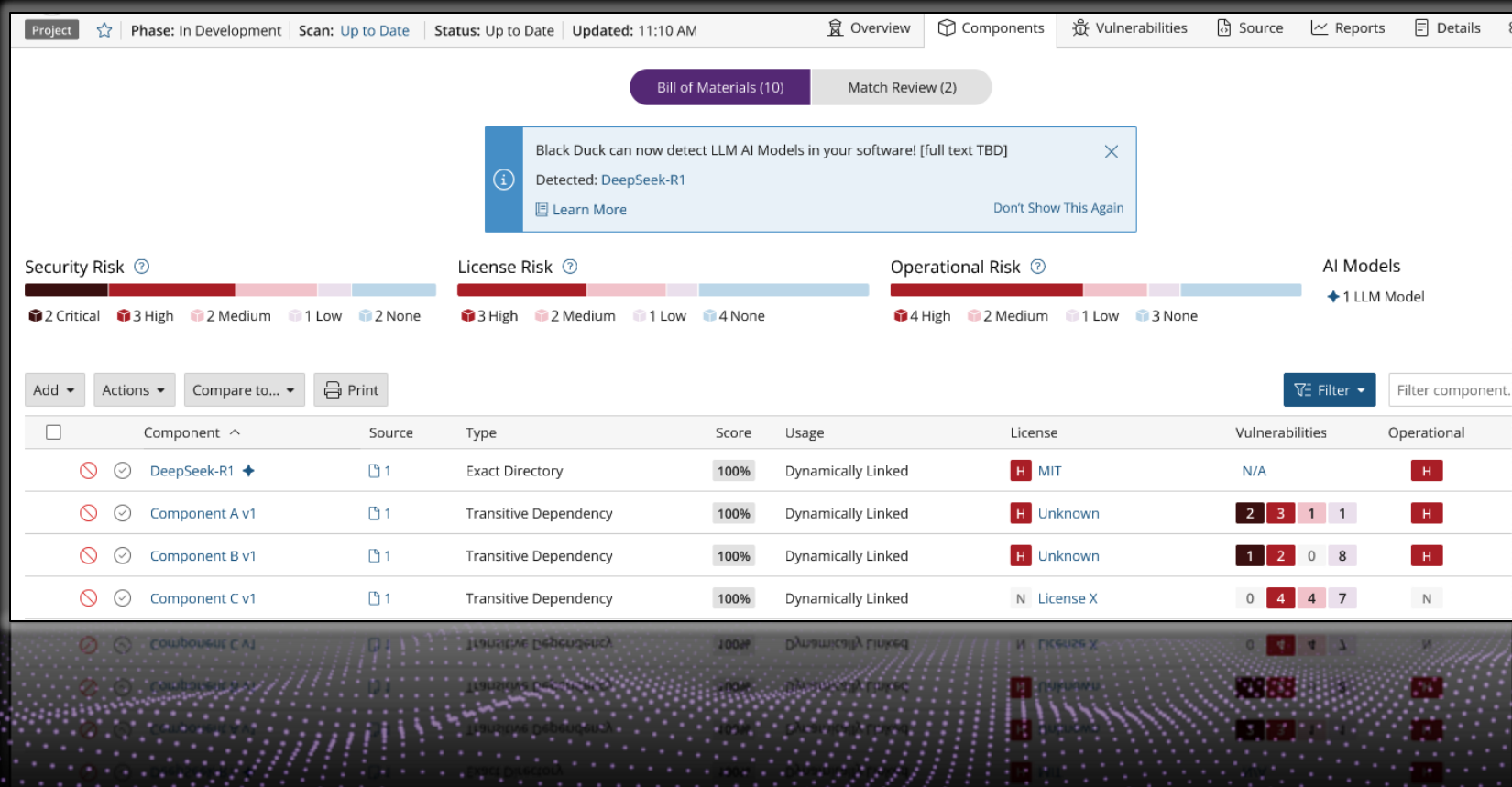


SCA가 오픈 소스
구성요소에 하는 일을
AI 모델에서도 합니다.

위험 완화:

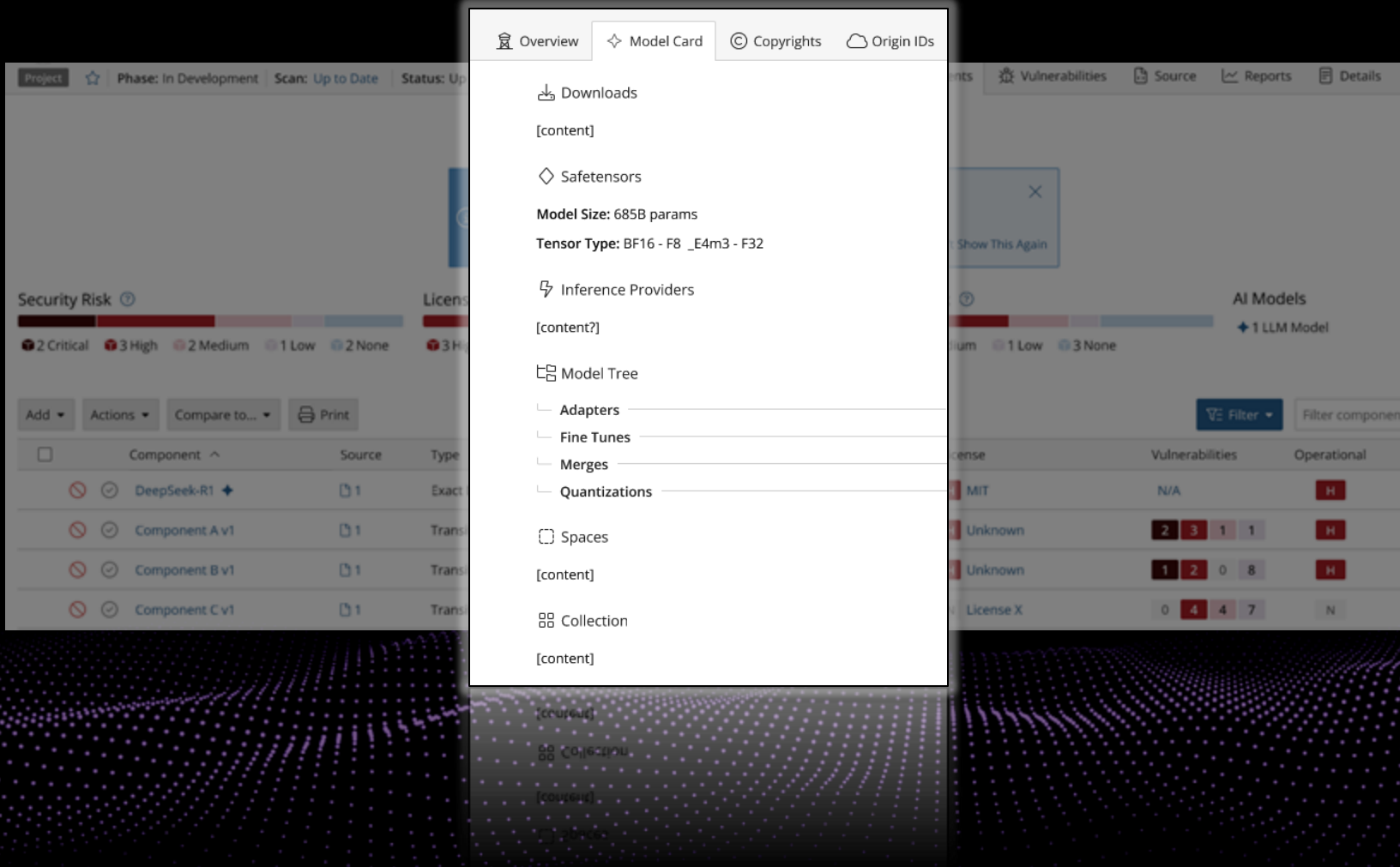
- AI 모델 조작
- 오염된 학습 데이터
- 의도적인 범위 변경

애플리케이션 코드를 넘어 AI 모델로 확장



- Hugging Face에서 제공된 모델의 탐지
- 기존 워크플로우 내에서 모델 탐지 및 식별
- 프로젝트에서 신뢰할 수 있고 평판이 좋은 모델의 사용
- 허용된 모델 관련 데이터의 주요 변경 사항에 대한 알림
- AI-BOMs 생성

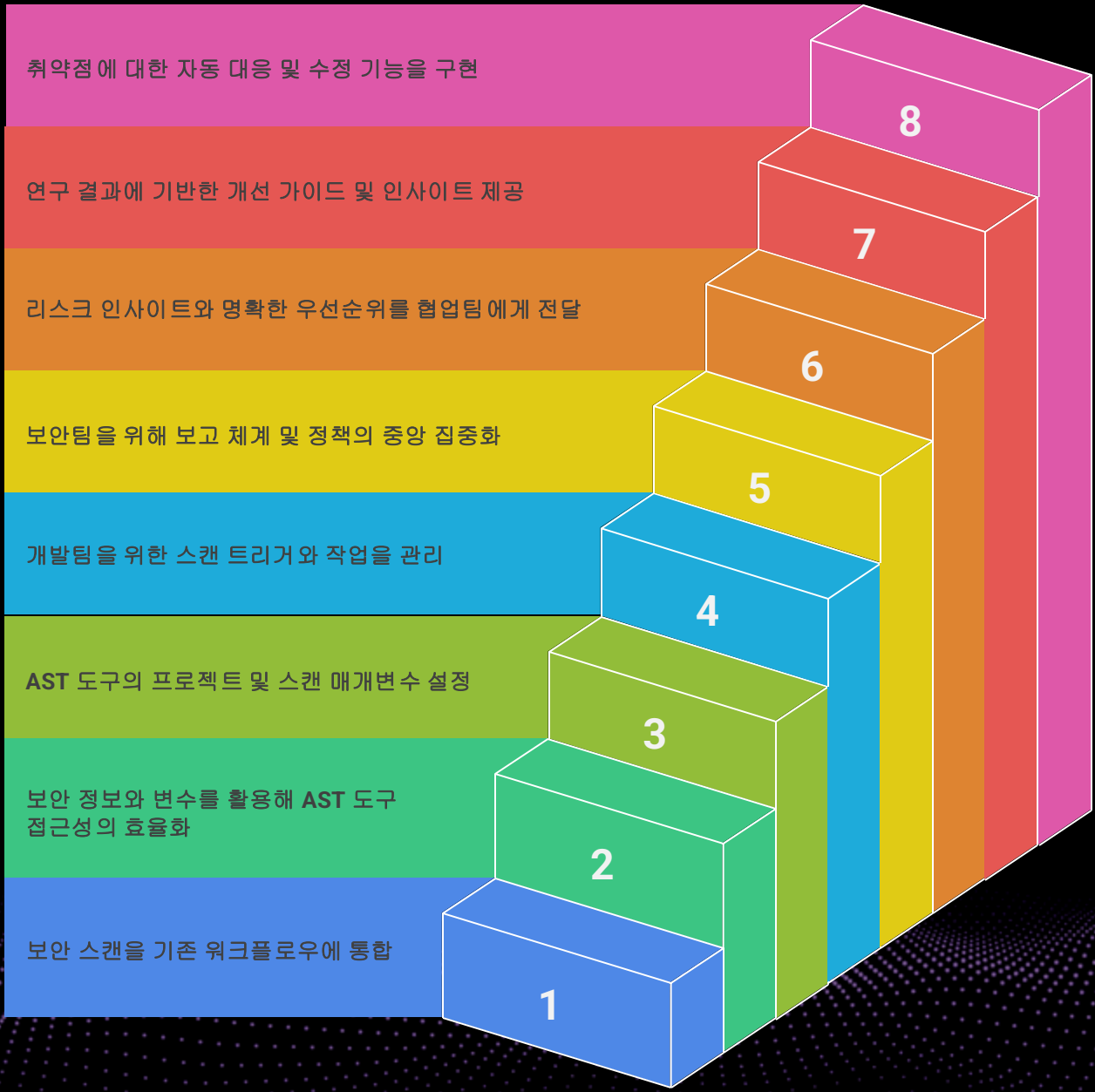
애플리케이션 코드를 넘어 **AI 모델**로 확장



- Hugging Face에서 제공된 모델을 빠르게 탐지하고 확인
- 기존 워크플로우 내에서 모델 탐지 및 식별
- 프로젝트에서 신뢰할 수 있고 평판이 좋은 모델을 사용
- 허용된 모델 관련 데이터의 주요 변경 사항에 대한 알림
- AI-BOMs 생성

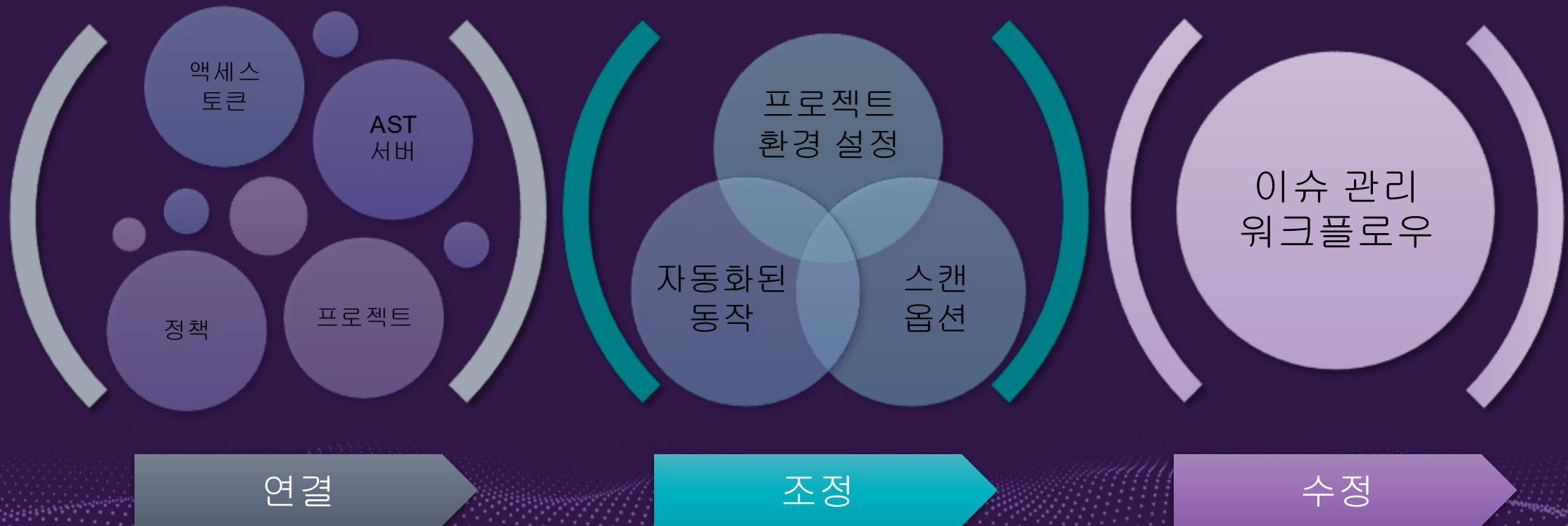
개발자의 빠르고 자신 있는 코드 배포를 위한 전략

AI 기반 개발을 위한 안전망 구축



AI 기반 파이프라인에 AppSec을 내재화하고 자동화하는 8단계 요약

개발자가 문제점을 빠르게 수정 할 수 있도록 돕는 3단계



AI 기반에서의 개발을 위한 AI 기반 AppSec

- DevOps와 CI 플랫폼들이 코드 생성과 자동 수정 기능을 위한 AI 도구들을 빠르게 개발하고 있습니다.
- 개발과 보안이 직면한 과제:
 - AI 도구들은 성숙도와 학습 측면에서 차이
 - 플랫폼 워크플로우 구성
 - 제한된 보안 테스트

AI가 만든 문제를 완화하기 위한 AI 기반의 보안

코드 수정

개발자들의 코드 변경 사항 반영;
AI 생성 코드와 OSS

소프트웨어 구성요소 분석(SCA)

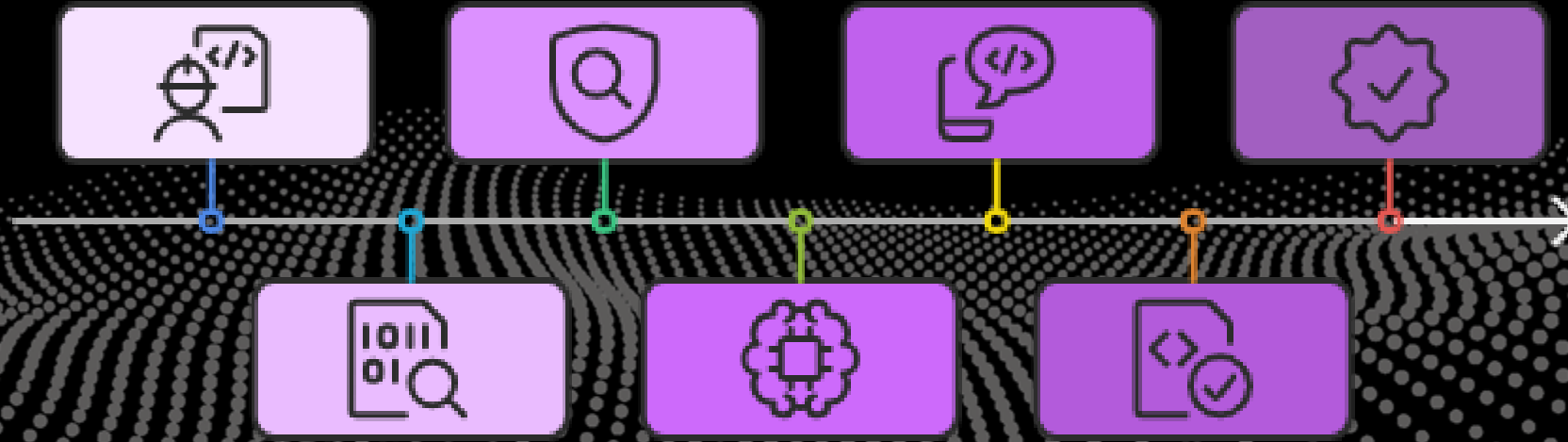
오픈 소스 취약점 탐지를 위한 코드
스캔

수정안의 개발자 전달

LLM이 판단한 수정안이 파이프라인
안에서 개발자에게 제시

애플리케이션 재테스트

검증된 수정사항을 적용한
애플리케이션의 재테스트 및
컴플라이언스 확인



정적 분석(SAST)

CWE와 결함 탐지를 위한 코드 스캔

LLM의 보안 수정안 결정

개선안을 찾기 위해 결함 있는
문제들이 LLM에 의해 분석

검증을 위한 코드 재스캔

해결은 잘 되었고 신규 이슈가
발생하지 않도록 코드 수정 사항을
재스캔



감사합니다.